

MEDICAL INFORMATION RAG ASSISTANT

Generative AI Based Medical Information Retrieval-Augmented Generation Assistant

A Lab Report submitted for “Generative AI”

Name: M Sai Ram Akash

Regd No: 231FA23058

Branch: ACSE-DS

Subject: Generative AI

Faculty: Absdul Rahman

Abstract

The Medical Information RAG Assistant is a Generative AI based educational system designed to provide reliable answers to general health-information questions using a controlled collection of trusted medical documents.

The system uses Retrieval-Augmented Generation (RAG), an approach that combines document retrieval with a Generative AI model. Instead of generating an answer only from the model's internal knowledge, the system first searches the available medical knowledge base for relevant information. The retrieved information is then provided to the Generative AI model as context for generating the final response.

The project is designed for educational and informational purposes. It does not diagnose patients, prescribe medicines, or provide personalized medical treatment recommendations. The use of a controlled document collection helps improve the relevance and traceability of generated responses.

The system demonstrates how Generative AI, document retrieval, embeddings, vector databases, and large language models can be combined to build a practical domain-specific question-answering application.

Contents

1	Introduction	5
2	Problem Statement	6
3	Objectives	7
4	Scope of the Project	8
5	Proposed System	9
6	System Architecture	10
6.1	Architecture Flow	10
6.2	User Interface	11
6.3	Knowledge Base	11
6.4	Document Processing	11
6.5	Vector Retrieval	11
6.6	Generative AI	11
7	Technologies Used	12
8	Methodology	13
8.1	Step 1: Requirement Analysis	13
8.2	Step 2: Knowledge Base Creation	13
8.3	Step 3: Document Loading	13
8.4	Step 4: Document Chunking	13
8.5	Step 5: Embedding Generation	14

8.6	Step 6: Vector Storage	14
8.7	Step 7: Query Processing	14
8.8	Step 8: Similarity Search	14
8.9	Step 9: Context Construction	14
8.10	Step 10: Response Generation	14
8.11	Step 11: Response Display	14
9	Implementation	15
9.1	Application Development	15
9.2	Streamlit Interface	15
9.3	Gemini API Integration	15
9.4	RAG Pipeline	16
10	Prompt Design	17
11	Functional Requirements	18
12	Non-Functional Requirements	19
13	Testing	20
14	Results	21
15	Advantages	22
16	Limitations	23
17	Future Scope	24
17.1	Larger Knowledge Base	24
17.2	Source Citations	24
17.3	Multilingual Support	24
17.4	Voice Interaction	24
17.5	Improved Retrieval	24
17.6	Document Management	25

17.7 Evaluation Framework	25
18 Ethical and Safety Considerations	26
19 Conclusion	27
20 References	28

Chapter 1

Introduction

Generative Artificial Intelligence has become an important area of modern Artificial Intelligence. Generative AI models are capable of understanding natural language and generating human-like responses based on user input.

Large Language Models can answer questions across many domains. However, when they are used in specialized domains such as healthcare, there is a need to provide reliable and controlled information. A model may generate information that is incomplete, outdated, or not directly supported by the available source material.

Retrieval-Augmented Generation provides a solution to this problem. RAG systems retrieve relevant information from an external knowledge base and provide that information to the language model before generating an answer.

The proposed Medical Information RAG Assistant applies this concept to general medical information. Users can enter educational health questions, and the system retrieves relevant information from a controlled collection of trusted medical documents before generating the response.

The application is intentionally designed as an educational information system rather than a medical diagnosis or treatment system.

Chapter 2

Problem Statement

General-purpose AI assistants can generate answers to health-related questions, but relying only on the model's internal knowledge can create challenges related to information accuracy, source relevance, and traceability.

Users may ask questions about symptoms, diseases, prevention, medications, medical terminology, or general health concepts. A system that provides unrestricted generated responses may produce information that is not supported by the intended knowledge source.

Therefore, there is a requirement for an educational assistant that:

- Uses a controlled collection of medical information.
- Retrieves relevant information before generating an answer.
- Uses Generative AI to produce understandable responses.
- Reduces dependence on unsupported model-generated information.
- Clearly communicates that the system is not a diagnostic tool.

The proposed system addresses these requirements using a Retrieval-Augmented Generation architecture.

Chapter 3

Objectives

The major objectives of the project are:

1. To develop an educational medical information assistant using Generative AI.
2. To implement a Retrieval-Augmented Generation pipeline.
3. To create a controlled medical document knowledge base.
4. To retrieve relevant document sections for a user's query.
5. To provide retrieved context to a Generative AI model.
6. To generate clear and understandable answers.
7. To reduce unsupported responses by grounding generation in retrieved information.
8. To demonstrate the practical application of Generative AI in a domain-specific system.

Chapter 4

Scope of the Project

The scope of the project is limited to providing general and educational health information.

The system can be used for:

- Understanding general medical concepts.
- Learning about diseases and health conditions.
- Understanding common medical terminology.
- Finding information from the provided document collection.
- Asking general educational health-related questions.

The system is not intended for:

- Medical diagnosis.
- Emergency medical decisions.
- Personalized treatment recommendations.
- Prescribing medicines.
- Replacing consultation with qualified healthcare professionals.

Chapter 5

Proposed System

The proposed Medical Information RAG Assistant consists of a document knowledge base, document processing module, embedding generation module, vector retrieval system, Generative AI model, and user interface.

When the user submits a question, the system processes the query and searches the vector knowledge base for relevant information. The retrieved document chunks are then combined with the user's question and passed to the Generative AI model.

The model generates an answer based on the retrieved context. This approach helps the system produce responses that are grounded in the information contained within the selected knowledge base.

The overall workflow is:

1. User enters a health-information question.
2. The question is converted into a suitable representation.
3. Relevant documents are searched.
4. The most relevant document chunks are retrieved.
5. Retrieved information is supplied as context.
6. Gemini Generative AI generates the response.
7. The response is displayed to the user.

Chapter 6

System Architecture

The system follows a Retrieval-Augmented Generation architecture.

The major components are:

1. User Interface
2. Query Processing Module
3. Document Knowledge Base
4. Document Chunking Module
5. Embedding Generation
6. Vector Database
7. Similarity Search
8. Context Construction
9. Gemini Generative AI Model
10. Response Generation

6.1 Architecture Flow

The architecture can be represented as:

User Query → Query Processing → Embedding → Vector Search → Relevant Documents → Context → Gemini → Generated Answer

6.2 User Interface

The user interface provides a simple mechanism for entering health-related questions and viewing generated responses.

6.3 Knowledge Base

The knowledge base contains selected medical documents that act as the primary information source for retrieval.

6.4 Document Processing

Documents are loaded and divided into smaller sections or chunks. Chunking makes it easier to retrieve only the relevant portions of a document.

6.5 Vector Retrieval

Each document chunk is represented using numerical embeddings. When the user asks a question, the query is also converted into an embedding.

The system compares the query representation with the stored document representations and retrieves the most relevant chunks.

6.6 Generative AI

The retrieved context is supplied to the Gemini Generative AI model. The model uses the context to generate a natural-language answer.

Chapter 7

Technologies Used

Technology	Purpose
Python	Main programming language used for application development.
Generative AI	Used to generate natural-language responses.
Gemini API	Provides access to the Generative AI model.
RAG	Combines retrieval with Generative AI generation.
Embeddings	Represents documents and queries in vector form.
Vector Database	Stores and searches document embeddings.
Streamlit	Provides the interactive web-based user interface.
Visual Studio Code	Development environment used for implementation.
Python Libraries	Used for document processing, retrieval, API integration, and application development.

Chapter 8

Methodology

The project follows a sequence of stages.

8.1 Step 1: Requirement Analysis

The first stage is to identify the requirements of the application. The system should answer general medical information questions using information from a controlled knowledge base.

8.2 Step 2: Knowledge Base Creation

A collection of trusted medical information documents is created. These documents provide the source material for the RAG system.

8.3 Step 3: Document Loading

The documents are loaded into the application using appropriate document-processing methods.

8.4 Step 4: Document Chunking

Large documents are divided into smaller chunks. Each chunk contains a manageable amount of information that can be retrieved independently.

8.5 Step 5: Embedding Generation

The document chunks are converted into numerical vector representations called embeddings.

8.6 Step 6: Vector Storage

The generated embeddings are stored in a vector database or vector-search structure.

8.7 Step 7: Query Processing

When the user enters a question, the system processes the query and generates its vector representation.

8.8 Step 8: Similarity Search

The query vector is compared with the document vectors. The system retrieves the document chunks that are most relevant to the question.

8.9 Step 9: Context Construction

The retrieved information is combined into a context that is sent to the Generative AI model.

8.10 Step 10: Response Generation

The Gemini model generates a response using the user's question and the retrieved context.

8.11 Step 11: Response Display

The generated answer is displayed through the Streamlit interface.

Chapter 9

Implementation

9.1 Application Development

The application is developed using Python. Python provides a large ecosystem of libraries for Artificial Intelligence, Natural Language Processing, document processing, and web application development.

9.2 Streamlit Interface

Streamlit is used to create the user interface.

The interface provides:

- Application title.
- Information about the system.
- Question input field.
- Submit functionality.
- Generated response area.
- Safety and educational-use information.

9.3 Gemini API Integration

The Gemini API is integrated into the application to provide Generative AI capabilities.

The application sends the user's question together with retrieved context to the model. The model then generates the final response.

The API key is stored securely using an environment variable rather than directly writing it into the source code.

9.4 RAG Pipeline

The RAG pipeline follows the following process:

```
graph TD; A[User Question] --> B[Query Processing]; B --> C[Query Embedding]; C --> D[Vector Similarity Search]; D --> E[Relevant Document Chunks]; E --> F[Context Construction]; F --> G[Gemini Generative AI]; G --> H[Final Answer];
```

User Question
|
v
Query Processing
|
v
Query Embedding
|
v
Vector Similarity Search
|
v
Relevant Document Chunks
|
v
Context Construction
|
v
Gemini Generative AI
|
v
Final Answer

Chapter 10

Prompt Design

Prompt engineering is an important part of the proposed system.

The prompt instructs the Generative AI model to use the retrieved context when generating its response.

The system prompt is designed around the following principles:

- Answer using the provided context whenever possible.
- Avoid inventing unsupported medical information.
- Clearly communicate uncertainty when the retrieved information is insufficient.
- Provide educational rather than diagnostic information.
- Avoid personalized medical treatment recommendations.
- Encourage users to consult qualified healthcare professionals for personal medical concerns.

An example instruction used conceptually by the system is:

```
You are an educational medical information assistant.
```

```
Use the retrieved context to answer the user's question.
```

```
Do not diagnose the user.
```

```
Do not prescribe medication or provide personalized  
treatment instructions.
```

```
If the available context does not contain enough information,  
clearly state that the information is insufficient.
```

```
Provide a clear, concise and educational response.
```

Chapter 11

Functional Requirements

The system should satisfy the following functional requirements:

1. The user should be able to enter a health-related question.
2. The system should process the user query.
3. The system should search the medical knowledge base.
4. The system should retrieve relevant information.
5. The retrieved information should be provided to the Generative AI model.
6. The model should generate an understandable response.
7. The system should display the generated response.
8. The system should maintain the educational nature of the application.

Chapter 12

Non-Functional Requirements

- **Usability:** The interface should be simple and understandable.
- **Reliability:** The application should handle common user queries consistently.
- **Performance:** Retrieval and response generation should be completed within a reasonable time.
- **Security:** API credentials should not be exposed in publicly accessible source code.
- **Maintainability:** The application should use modular and understandable code.
- **Scalability:** The knowledge base should be extendable with additional documents.

Chapter 13

Testing

Testing is performed to verify the functionality of the complete RAG pipeline.

Test Case	Input	Expected Result
TC01	General medical question	System retrieves relevant information and generates an answer.
TC02	Question related to a document in the knowledge base	Relevant document information is retrieved.
TC03	Question outside the knowledge base	System should indicate that sufficient information is not available.
TC04	Empty question	System should request a valid question.
TC05	General health terminology question	System should provide an educational explanation.
TC06	Personalized diagnosis request	System should avoid providing a diagnosis and direct the user toward professional medical advice.

Chapter 14

Results

The developed system demonstrates the integration of Retrieval- Augmented Generation with Generative AI for medical information retrieval.

The application is able to:

- Accept natural-language questions.
- Search the controlled knowledge base.
- Retrieve relevant information.
- Provide retrieved information as context to the AI model.
- Generate natural-language responses.
- Display the response through the user interface.

The RAG architecture provides a structured approach to domain-specific question answering because the generated response is informed by retrieved information from the configured knowledge base.

The system also demonstrates the importance of separating general health education from medical diagnosis and personalized treatment.

Chapter 15

Advantages

The major advantages of the proposed system are:

1. **Knowledge Grounding:** Responses are generated using retrieved information from the configured knowledge base.
2. **Domain Specificity:** The system can be focused on a particular collection of medical information.
3. **Natural Language Interaction:** Users can ask questions using ordinary language.
4. **Expandable Knowledge Base:** Additional documents can be added to the system.
5. **Educational Use:** The system can help users understand general medical concepts.
6. **Generative AI Integration:** The project demonstrates a practical application of a modern Generative AI model.

Chapter 16

Limitations

The system has several limitations:

1. The quality of the response depends on the quality of the documents in the knowledge base.
2. Retrieval errors can affect the generated response.
3. Generative AI models may still produce incorrect information.
4. The system is not a replacement for professional healthcare advice.
5. API availability and network connectivity may affect system operation.
6. The system does not independently verify every generated statement against a medical professional's assessment.

Chapter 17

Future Scope

The project can be extended in several ways.

17.1 Larger Knowledge Base

The system can be extended with a larger collection of carefully selected medical documents.

17.2 Source Citations

Future versions can display the specific document sources used to generate each response.

17.3 Multilingual Support

The application can be extended to support multiple Indian languages, allowing users to ask questions in regional languages.

17.4 Voice Interaction

Speech-to-text and text-to-speech capabilities can be added to allow voice-based interaction.

17.5 Improved Retrieval

Advanced hybrid retrieval methods combining keyword and semantic search can be implemented.

17.6 Document Management

An administrative interface can be developed for adding, removing, and updating knowledge-base documents.

17.7 Evaluation Framework

A dedicated evaluation system can be introduced to measure retrieval accuracy, response relevance, factual consistency, and response quality.

Chapter 18

Ethical and Safety Considerations

Healthcare information requires careful handling because incorrect information can potentially cause harm.

The proposed project therefore follows an educational-use approach.

The system should not:

- Diagnose a user's medical condition.
- Prescribe medication.
- Recommend personalized treatment.
- Replace professional medical consultation.
- Present generated information as a confirmed medical diagnosis.

The system should clearly communicate its limitations to users.

For personal medical concerns, users should consult an appropriately qualified healthcare professional.

Chapter 19

Conclusion

The Medical Information RAG Assistant demonstrates the practical application of Generative AI and Retrieval-Augmented Generation in a domain-specific information system.

The project combines document processing, embeddings, vector retrieval, contextual prompting, and the Gemini Generative AI model to produce educational responses to general medical-information questions.

The RAG approach provides an important architectural advantage by allowing the Generative AI model to use information retrieved from an external knowledge base rather than relying exclusively on its internal model knowledge.

The project also highlights the importance of responsible AI development in healthcare-related applications. Since the system is intended for educational information rather than diagnosis or treatment, appropriate safety boundaries are maintained.

Overall, the project provides practical experience with modern Generative AI concepts and demonstrates how RAG systems can be used to develop useful domain-specific AI assistants.

Chapter 20

References

1. Google, “Gemini API Documentation,” Google AI for Developers.
2. Streamlit, “Streamlit Documentation,” Streamlit Documentation.
3. Python Software Foundation, “Python Documentation.”
4. Lewis, P. et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.”
5. Relevant medical information documents used to construct the controlled knowledge base.
6. Relevant research papers and technical documentation related to Retrieval-Augmented Generation, embeddings, vector search, and Generative AI.